# CultivaX — User Manual

**Version**: 3.0 · **Date**: April 2026 · **B.Tech 4th Semester Project**

---

# PART 1 — QUICK REFERENCE GUIDE

This section gives a concise overview of the CultivaX platform so you can understand what it does and start using it quickly.

---

## 1.1 What is CultivaX?

CultivaX is a cloud-based agricultural management platform designed for Indian farmers, service providers, and platform administrators. It helps farmers track their crops from sowing to harvest, connect with equipment and labor providers, and access government schemes — all from a single web application available in 5 languages.

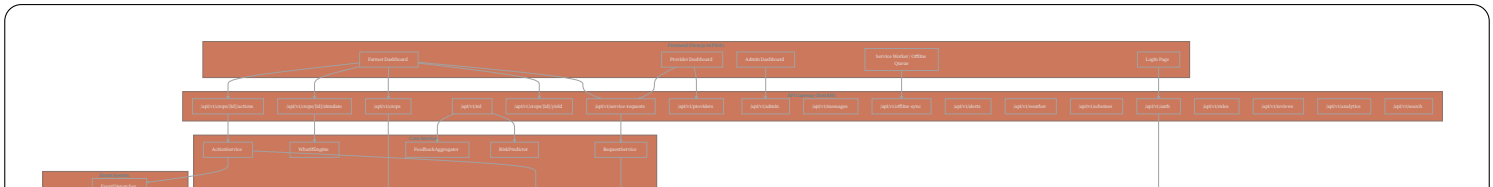
The platform is built around three core systems:

| Abbreviation | Full Name | What It Does |
|---|---|---|
| **CTIS** | Crop Timeline Integration System | Tracks every crop activity (irrigation, fertilizer, etc.) in a time-ordered log and automatically calculates crop health scores and risk levels |
| **SOE** | Service Orchestration Engine | A marketplace that lets farmers book equipment and labor from registered service providers |
| **IAM** | Identity & Access Management | Handles login, OTP verification, user roles, and account security |

CultivaX is accessible as a **Progressive Web App (PWA)**, which means it can be used offline and installed on a phone or desktop like a native app. It supports **English, Hindi, Tamil, Telugu, and Marathi**.

---

## 1.2 System Architecture Overview



---

## 1.3 Demo Credentials

All demo accounts use the password: `password123`

| Role | Phone Number | What They Can Do |
|---|---|---|
| **Farmer** | `+919876543210` | Create crops, log actions, run simulations, submit yields, browse services, send messages |
| **Service Provider** | `+919876543211` | View, accept, and complete service requests; manage equipment; view reviews |
| **Platform Admin** | `+919876543212` | Manage users, handle failed events, oversee ML models, review abuse flags, view audit logs |

> **Note for Evaluators**: Login sessions last **4 hours** before expiring. The rate limit is set to **100 requests/minute**, so you will not be locked out during normal evaluation use. In development mode, OTPs are returned directly in the API response — no real SMS is sent.

---

## 1.4 How to Start the Application

### Option A: Docker (Recommended)

```bash
git clone <repo-url> && cd CultivaX
cp .env.example .env
docker compose up --build
```

Once running, open these URLs in your browser:

- **Web App:** `http://localhost:3000`
- **API:** `http://localhost:8000`
- **API Documentation (Swagger):** `http://localhost:8000/docs`

### Option B: Manual Setup (Developer Machine)

```bash
```

```
# Backend
cd backend
python3.11 -m venv venv && source venv/bin/activate
pip install -r requirements.txt
alembic upgrade head
python -m scripts.seed_demo_users
uvicorn app.main:app --reload --port 8000

# Frontend (separate terminal)
cd frontend
npm install && npm run dev
```

---

## 1.5 Feature Map — All 40 Modules

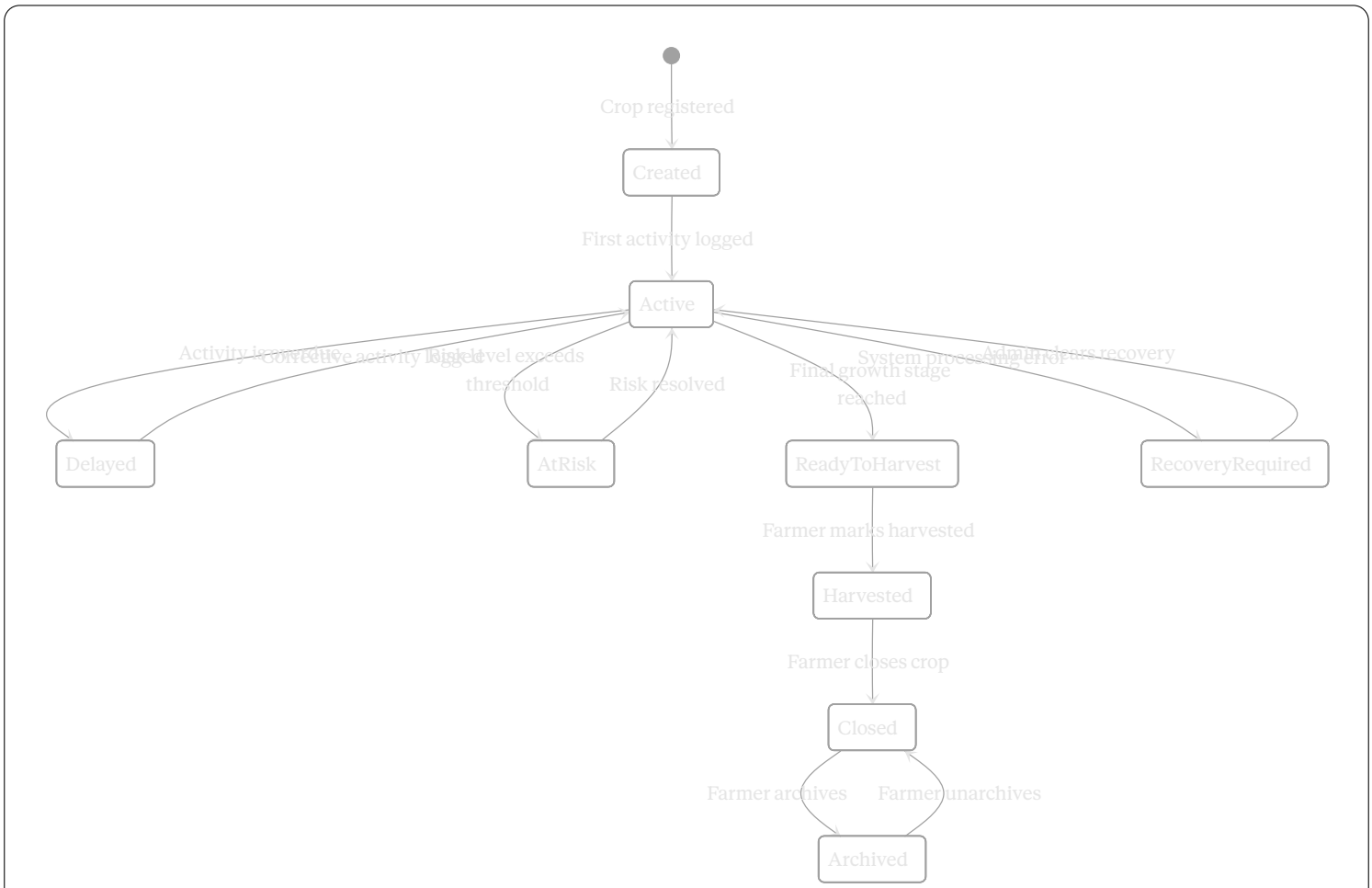
| # | Module | Who Uses It | API Prefix | Page |
|---|---|---|---|---|
| 1 | Login / Register / OTP | All | `/auth` | `/login`, `/register` |
| 2 | Farmer Dashboard | Farmer | `/dashboard` | `/dashboard` |
| 3 | Crop Management | Farmer | `/crops` | `/crops`, `/crops/new` |
| 4 | Action Logging (CTIS) | Farmer | `/crops/{id}/actions` | `/crops/[id]/log-action` |
| 5 | Crop Timeline & History | Farmer | `/crops/{id}/actions` | `/crops/[id]/history` |
| 6 | Replay Engine & Snapshots | Farmer | `/crops/{id}/replay/*` | `/crops/[id]` |
| 7 | What-If Simulation | Farmer | `/crops/{id}/simulate` | `/crops/[id]/simulat` |
| 8 | Yield Submission | Farmer | `/crops/{id}/yield` | `/crops/[id]/yield` |
| 9 | Weather & Risk | Farmer | `/weather` | `/weather` |
| 10 | Crop Recommendations | Farmer | `/crops/{id}/recommendations` | `/crops/[id]` |
| 11 | Alerts & Notifications | Farmer | `/alerts` | `/alerts` |

| | | | | |
|---|---|---|---|---|
| 12 | Government Schemes | Farmer | `/schemes` | `/schemes` |
| 13 | Service Marketplace | Farmer | `/service-requests` | `/services` |
| 14 | Service Request Lifecycle | Farmer / Provider | `/service-requests/{id}/*` | `/services/my-requests` |
| 15 | Service Reviews | Farmer | `/reviews` | `/services/review` |
| 16 | In-App Messaging | Farmer / Provider | `/messages` | `/messages` |
| 17 | Provider Dashboard | Provider | `/providers` | `/provider` |
| 18 | Equipment Management | Provider | `/providers/{id}/equipment` | `/provider/equipment` |
| 19 | Labor Management | Provider | `/labor` | `/labor` |
| 20 | Provider Onboarding | Provider | `/onboarding` | `/provider/onboardin` |
| 21 | Provider Reviews | Provider | `/reviews` | `/provider/reviews` |
| 22 | Admin Dashboard | Admin | `/admin` | `/admin` |
| 23 | User Management | Admin | `/admin/users` | `/admin/users` |
| 24 | Provider Governance | Admin | `/admin/providers` | `/admin/providers` |
| 25 | Dead Letter Queue | Admin | `/admin/dead-letters` | `/admin/dead-letters` |
| 26 | Audit Trail | Admin | `/admin/audit` | `/admin/audit` |
| 27 | ML Model Management | Admin | `/ml` | `/admin/ml-models` |
| 28 | Crop Rules Engine | Admin | `/rules` | `/admin/rules` |
| 29 | Feature Flags | Admin | `/features` | `/admin/features` |
| 30 | System Health | Admin | `/admin/health` | `/admin/health` |
| 31 | Maintenance / | Admin | `/admin/maintenance` | `/admin/maintenance` |

| | | | | |
|---|---|---|---|---|
| | Cron | | | |
| 32 | Security Events | Admin | `/admin/security-events` | `/admin/security` |
| 33 | Analytics | Admin | `/analytics` | `/admin/analytics` |
| 34 | Reports & Disputes | Admin | `/reports`, `/disputes` | `/admin/reports`, `/disputes` |
| 35 | Offline Sync | Farmer | `/offline-sync` | (PWA Service Worker) |
| 36 | Consent Management | All | `/consent` | `/settings/consent` |
| 37 | Account Settings | All | `/account` | `/settings/account` |
| 38 | Land Parcels | Farmer | `/land-parcels` | `/land-parcels` |
| 39 | Search | All | `/search` | (Global search bar) |
| 40 | Language | All | `/translations` | (Language picker) |

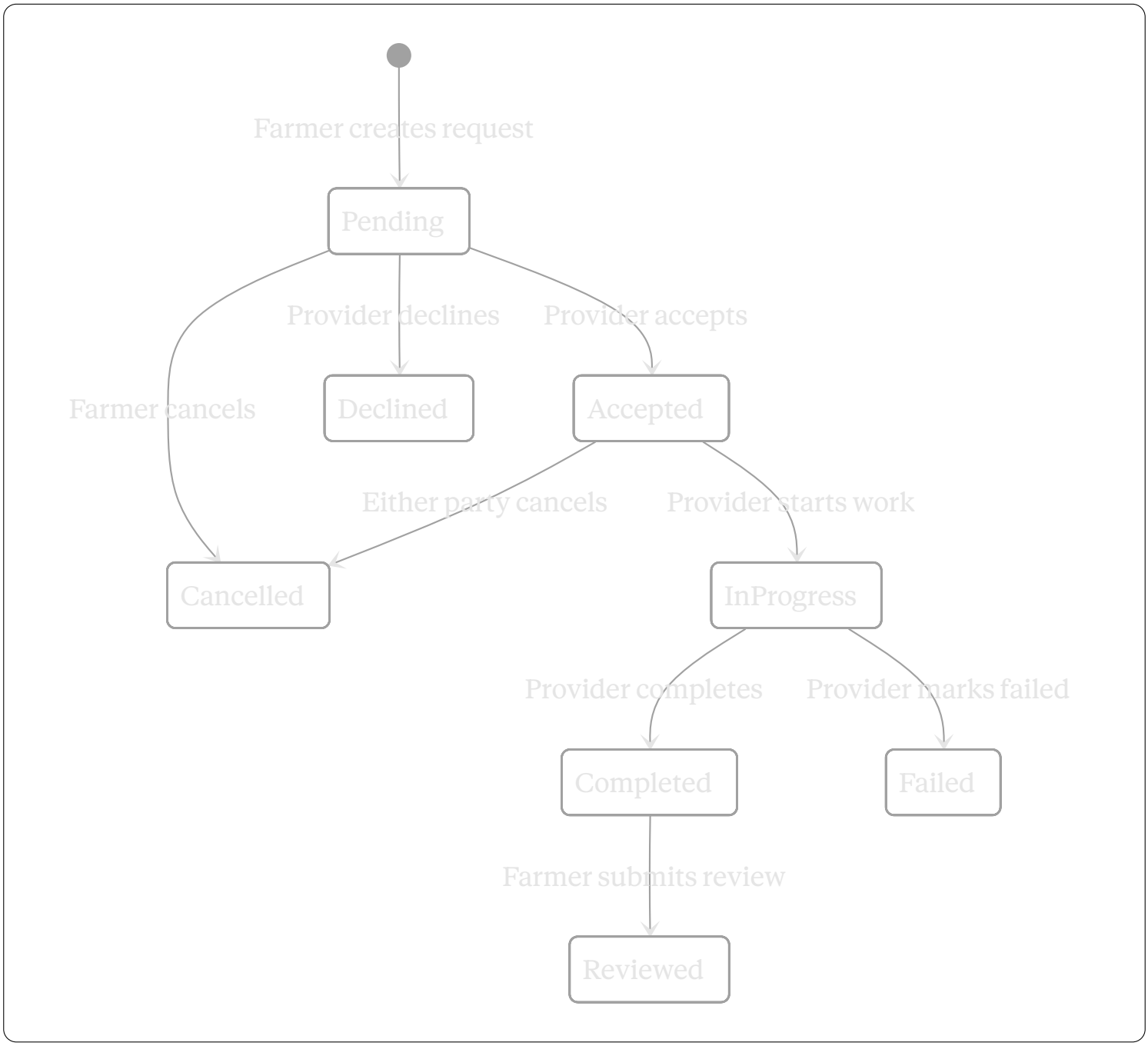
## 1.6 Crop Status Reference

A crop moves through the following statuses during its lifetime:



## 1.7 Service Request Status Reference

A service request moves through these statuses:




# PART 2 — DETAILED STEP-BY-STEP GUIDE

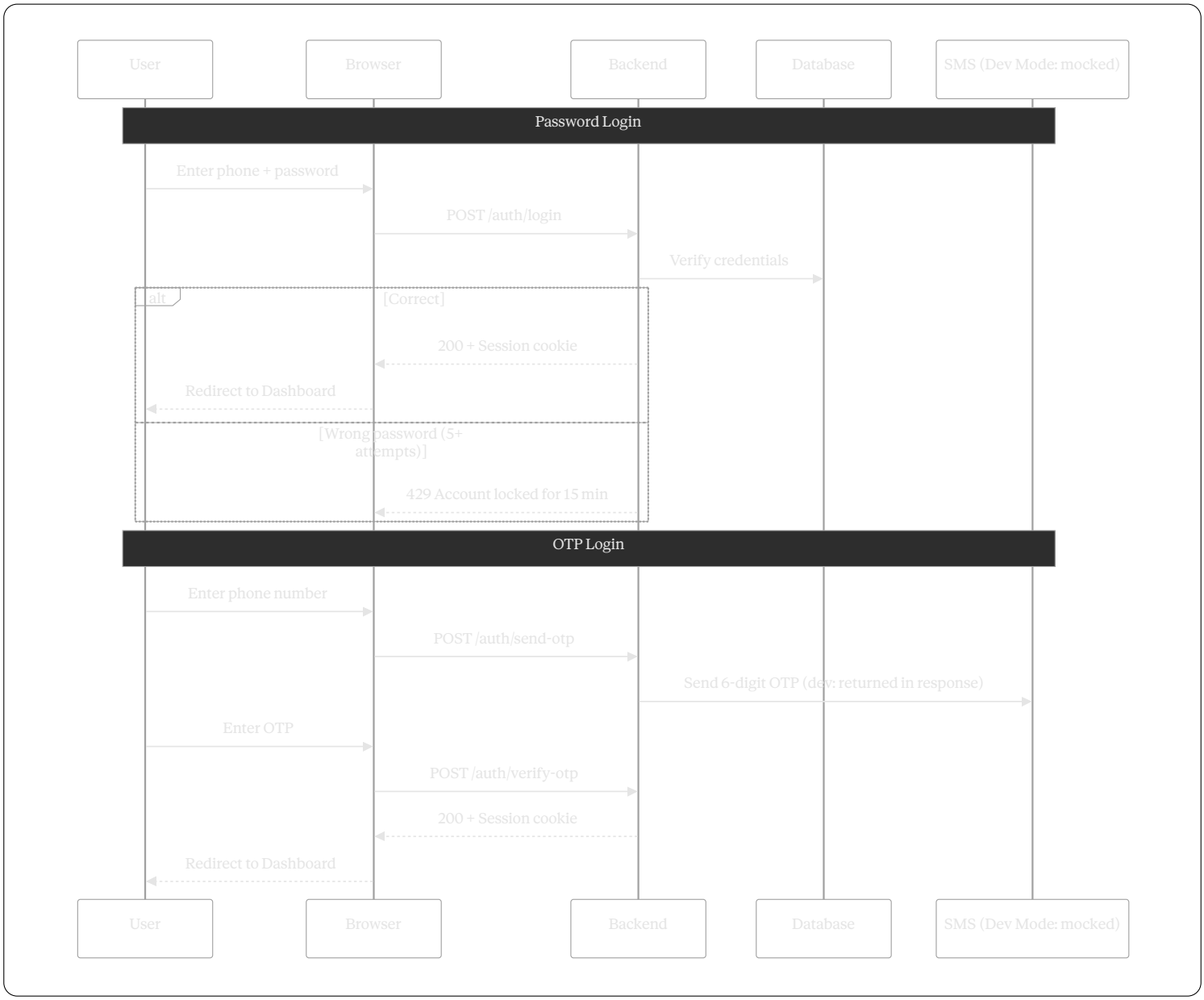
This section provides complete instructions for every feature, including what each feature does, exactly how to use it, all input fields, and expected results.

## 2.1 Authentication

### 2.1.1 Overview

CultivaX uses phone-based authentication with two options: password login or one-time password (OTP). All sessions are secured with JWT tokens stored in HttpOnly cookies (protected from browser script attacks). Accounts are automatically locked for 15 minutes after 5 consecutive failed login attempts.

### 2.1.2 Full Authentication Flow




### 2.1.3 Creating an Account

**Endpoint:** `POST /api/v1/auth/register`

**Required Fields:**

| Field | Type | Rules | Example |
|---|---|---|---|
| `full_name` | text | 1–200 characters | `"Rajesh Kumar"` |
| `phone` | text | Indian phone format | `"+919876543210"` |
| `password` | text | Minimum 8 characters | `"password123"` |
| `role` | text | `farmer` or `provider` only | `"farmer"` |

| | | | |
|---|---|---|---|
| `region` | text | 1–100 characters | `"Punjab"` |

**Optional Fields:** `email`, `preferred_language` (default: `en`)

**Important Rules:**

- Admin accounts cannot be self-registered
- Duplicate phone numbers return a `409 Conflict` error
- Passwords are stored using Argon2id hashing

**Steps (using the web app):**

1. Go to `http://localhost:3000/register`
2. Fill in your Full Name, Phone Number, Password, select your Role (Farmer or Provider), and enter your Region
3. Click **Register**
4. On success, you are automatically signed in and taken to your dashboard

### 2.1.4 Password Login

**Endpoint:** `POST /api/v1/auth/login`

**Steps:**

1. Go to `http://localhost:3000/login`
2. Enter phone number and password
3. Click **Login**
4. On success, you are redirected to your dashboard

**Error Reference:**

| Situation | Response Code | Message |
|---|---|---|
| Phone number not found | 401 | "Invalid phone number or password" |
| Wrong password | 401 | "Invalid phone number or password" |
| 5 or more failed attempts | 429 | "Account locked. Try again in N seconds" |
| Account deactivated | 403 | "Account is deactivated. Contact admin." |

### 2.1.5 OTP Login

**Step 1 — Request OTP:** `POST /api/v1/auth/send-otp`

**Step 1 — Request OTP:** `POST /api/v1/auth/send-otp`

- Body: `{ "phone": "+919876543210" }`
- Response: `{ "message": "OTP sent successfully", "expires_in_seconds": 300, "debug_otp": "123456" }`
- The `debug_otp` field in development mode contains the actual OTP for testing. In production, it is sent via SMS.
- OTP expires in **5 minutes**. Maximum **5 OTP requests per phone per hour**.

**Step 2 — Verify OTP:** `POST /api/v1/auth/verify-otp`

- Body: `{ "phone": "+919876543210", "otp": "123456" }`
- On success, you are signed in

### 2.1.6 Token Refresh

**Endpoint:** `POST /api/v1/auth/refresh`

Automatically refreshes your session using the stored cookie. Each refresh rotates both the access and refresh tokens. If an old refresh token is reused, all sessions for that account are immediately revoked as a security measure.

### 2.1.7 Logout

**Endpoint:** `POST /api/v1/auth/logout`

Ends your current session, clears all cookies, and revokes the session record.

### 2.1.8 Session Management

- **View all sessions:** `GET /api/v1/auth/sessions` — shows each active session with device, IP address, and browser
- **Revoke all other sessions:** `POST /api/v1/auth/sessions/revoke-all` — signs out all devices except the current one

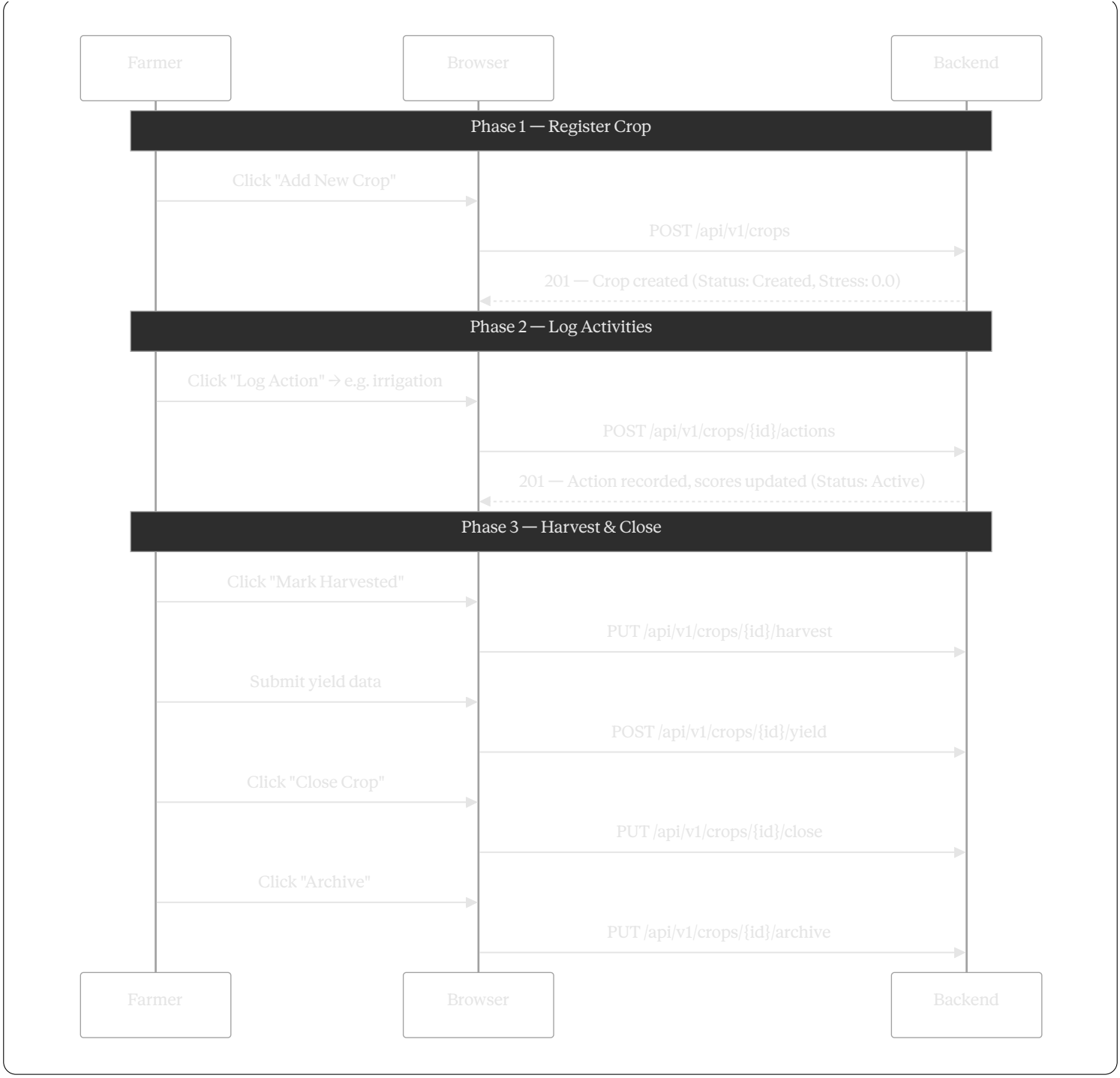
---

## 2.2 Crop Lifecycle Management (CTIS)

### 2.2.1 Overview

The Crop Timeline Integration System (CTIS) is the core of CultivaX. It records every activity on a crop — irrigation, fertilizer, observation, pesticide, harvesting — as a permanent, time-ordered log. After each activity is recorded, the system automatically recalculates the crop's health score, risk level, and growth stage.

### 2.2.2 Complete Crop Lifecycle Flow




### 2.2.3 Creating a Crop

**Endpoint:** `POST /api/v1/crops/`

**Required Fields:**

| Field | Type | Rules | Example |
|---|---|---|---|
| `crop_type` | text | 1–100 characters | `"wheat"` |
| `sowing_date` | date | ISO 8601 format | `"2026-03-01"` |
| `region` | text | 1–100 characters | `"Punjab"` |

**Optional Fields:**

| Field | Type | Example |
|---|---|---|

| | | |
|---|---|---|
| `variety` | text | `"HD-3226"` |
| `land_area` | number (> 0) | `5.5` (acres) |
| `sub_region` | text | `"Ludhiana"` |
| `land_parcel_id` | UUID | Link to a saved land parcel |
| `rule_template_id` | UUID | Use a specific crop rule template |
| `metadata_extra` | JSON | `{"soil_type": "loamy"}` |

**Steps:**

1. Log in as Farmer (`+919876543210`)
2. Go to `/crops` → **My Crops**
3. Click **Add New Crop** (opens `/crops/new`)
4. Fill in Crop Type, Sowing Date (pick a date at least 30 days in the past to log activities), and Region
5. Optionally add Variety, Land Area, Land Parcel
6. Click **Create Crop**
7. You are redirected to the crop detail page

**Response Fields:**

| Field | Description |
|---|---|
| `id` | Unique crop ID |
| `state` | `"Created"` (initial status) |
| `stress_score` | `0.0` (starts at zero) |
| `risk_index` | `0.0` (starts at zero) |
| `seasonal_window_category` | `"Early"`, `"Optimal"`, or `"Late"` — calculated from sowing date and region |
| `max_allowed_drift` | `7` — default tolerance in days before a crop is marked Delayed |

### 2.2.4 Listing Crops

**Endpoint:** `GET /api/v1/crops/?page=1&per_page=20`

**Available Filters:**

| Filter | Description | Example |
|---|---|---|
| `state` | Filter by status | `state=Active` |
| `crop_type` | Filter by crop | `crop_type=wheat` |
| `region` | Filter by region | `region=Punjab` |
| `include_archived` | Include archived crops | `include_archived=true` |
| `search` | Text search | `search=HD-3226` |
| `seasonal_window_category` | Filter by sowing window | `seasonal_window_category=Optimal` |

### 2.2.5 Updating a Crop

**Endpoint:** `PUT /api/v1/crops/{crop_id}`

**Fields you can update:** `variety`, `land_area`, `land_parcel_id`, `sub_region`, `metadata_extra`

> `crop_type`, `sowing_date`, `state`, `stress_score`, and `risk_index` cannot be updated directly. Status changes go through the status workflow, and scores are recalculated automatically.

### 2.2.6 Changing the Sowing Date

**Endpoint:** `PUT /api/v1/crops/{crop_id}/sowing-date`

**Body:** `{ "new_sowing_date": "2026-02-15" }`

When the sowing date is changed, all crop health scores and metrics are fully recalculated from scratch using the new date as the baseline.

### 2.2.7 Crop Status Transitions

| Action | Endpoint | From Status | To Status |
|---|---|---|---|
| First activity logged | (automatic) | Created | Active |
| Mark harvested | `PUT /crops/{id}/harvest` | Active, ReadyToHarvest | Harvested |
| Close crop | `PUT /crops/{id}/close` | Harvested | Closed |

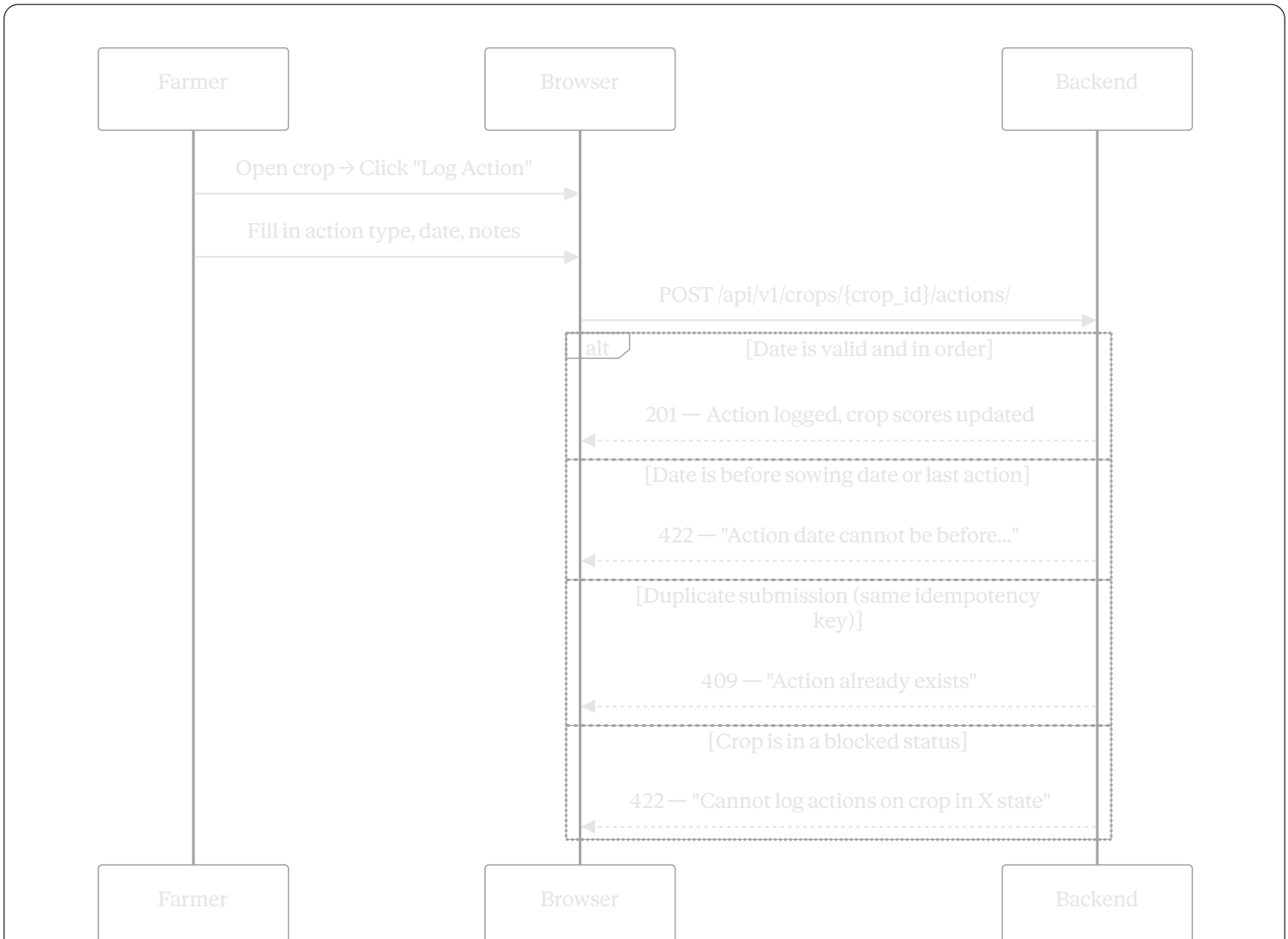
| | | | |
|---|---|---|---|
| Archive | `PUT /crops/{id}/archive` | Closed | Archived |
| Unarchive | `PUT /crops/{id}/unarchive` | Archived | Closed |
| Admin clear recovery | `PATCH /crops/{id}/_admin/recovery/clear` | RecoveryRequired | Active |
| Admin force replay | `PATCH /crops/{id}/_admin/recovery/retry` | Any | Active |

## 2.3 Logging Crop Activities

### 2.3.1 Overview

Every action a farmer takes on a crop — irrigation, fertilizer application, pesticide, observation, or harvesting — is recorded with a date. The system enforces a strict rule: activities must always be recorded in chronological order. No activity can be dated before a previously logged one.

### 2.3.2 Activity Logging Flow



### 2.3.3 Logging an Activity

**Endpoint:** `POST /api/v1/crops/{crop_id}/actions/`

**Required Fields:**

| Field | Type | Rules | Example |
|---|---|---|---|
| `action_type` | text | One of the supported types | `"irrigation"`, `"fertilizer"`, `"observation"`, `"pesticide"`, `"harvesting"` |
| `effective_date` | date | Must be on or after sowing date AND on or after the last logged activity | `"2026-03-15"` |

**Optional Fields:**

| Field | Type | Default | Example |
|---|---|---|---|
| `category` | text | `"Operational"` | `"Timeline-Critical"` or `"Corrective"` |
| `metadata_json` | JSON | `{}` | `{"weather_risk": 0.3, "amount_liters": 500}` |
| `notes` | text | none | `"Applied urea 50kg/acre"` |
| `local_seq_no` | number | none | `1` (for ordering offline submissions) |
| `device_timestamp` | datetime | none | Offline device clock time |
| `idempotency_key` | text | none | `"dedup-abc123"` (prevents duplicate submissions) |

**Steps:**

1. Go to `/crops/[id]` (your crop detail page)
2. Click **Log Action** (opens `/crops/[id]/log-action`)
3. Select an Action Type (e.g., `irrigation`)
4. Pick the Effective Date — must be on or after the sowing date and on or after any previously logged activity
5. Add optional Notes (e.g., "Applied 500 liters via drip system")
6. Add optional metadata JSON if needed

6. Add optional metadata as JSON if needed
7. Click **Submit Action**
8. On success, you are redirected back to the crop detail with the updated timeline

**Response Fields:**

| Field | Description |
|---|---|
| `id` | Unique ID of the logged activity |
| `crop_instance_id` | ID of the parent crop |
| `action_type` | The type you submitted |
| `effective_date` | The confirmed date |
| `category` | Operational / Timeline-Critical / Corrective |
| `is_override` | Whether this overrides a recommended action |
| `applied_in_replay` | Calculation status |
| `server_timestamp` | When the server received the submission |

### 2.3.4 Viewing Activity History

**Endpoint:** `GET /api/v1/crops/{crop_id}/actions/?page=1&page_size=20&sort=-effective_date`

Sort options: `effective_date` (oldest first) or `-effective_date` (newest first, default).

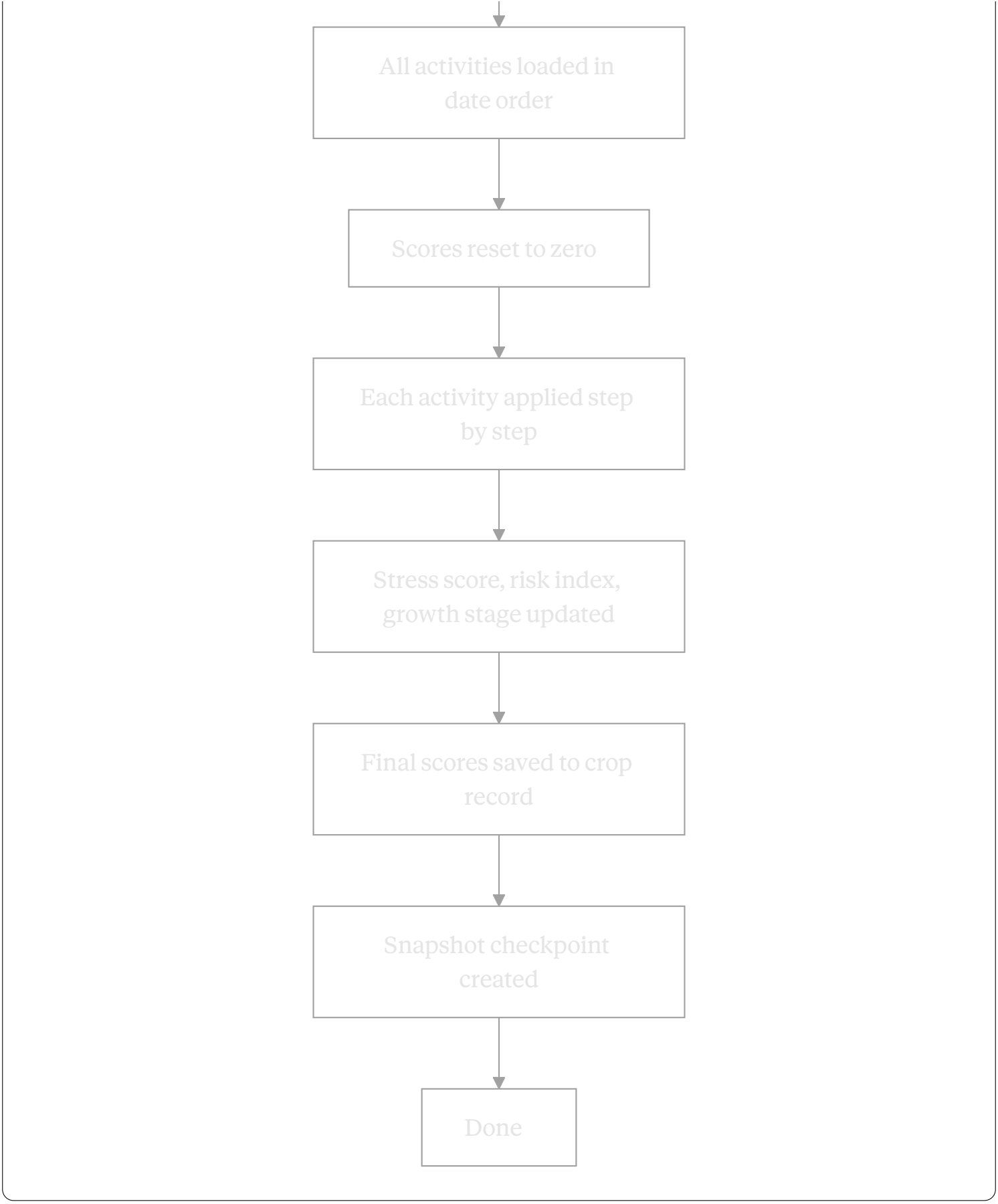
---

## 2.4 Crop Health Scores & Snapshots

### 2.4.1 Overview

Every time an activity is logged, CultivaX recalculates your crop's health by replaying the entire activity timeline from the beginning. This ensures that the stress score, risk index, and growth stage are always accurate and consistent. Periodic snapshots are saved as checkpoints so the system can recover quickly if needed.

### 2.4.2 How Recalculation Works




### 2.4.3 Checking Replay Status

**Endpoint:** `GET /api/v1/crops/{crop_id}/replay/status`

| Field | Description |
|---|---|
| `status` | `"idle"` (normal) or `"blocked"` (needs admin attention) |
| `last_replay_at` | When scores were last recalculated |

| | |
|---|---|
| `last_replay_at` | When scores were last recalculated |
| `actions_replayed_since_last_snapshot` | Activities processed since the last checkpoint |
| `recovery_required` | `true` if the crop needs admin recovery |
| `recovery_reason` | Error message if calculation failed |
| `snapshot_count` | Total snapshots saved for this crop |
| `latest_snapshot_id` | Most recent snapshot ID |

### 2.4.4 Viewing Calculation History

**Endpoint:** `GET /api/v1/crops/{crop_id}/replay/history?limit=50`

Returns the log of all recalculation events for the crop, including any failures and recoveries.

### 2.4.5 Viewing Snapshots

- **List all:** `GET /api/v1/crops/{crop_id}/snapshots?page=1&page_size=20`
- **View one:** `GET /api/v1/crops/{crop_id}/snapshots/{snapshot_id}`

Each snapshot contains: `stress_score`, `risk_index`, `stage`, `chain_hash`, `action_index`.

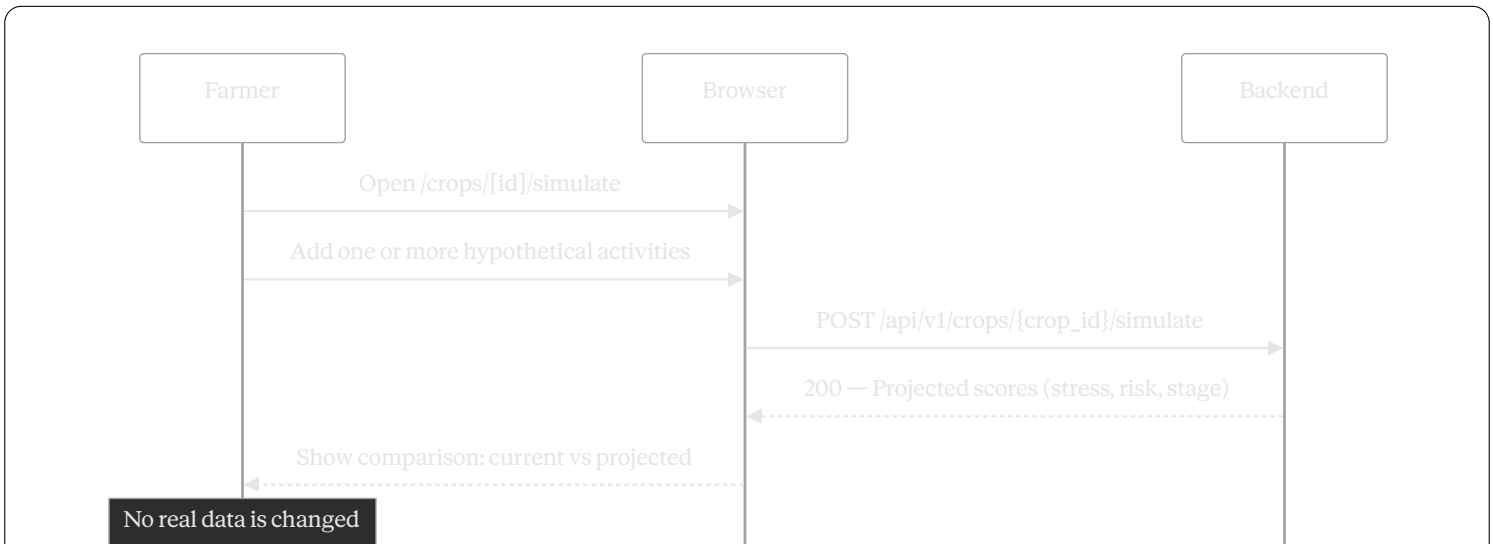
---

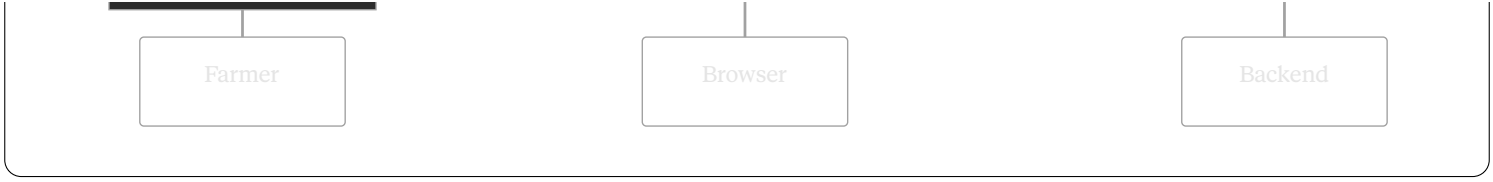
## 2.5 What-If Simulation

### 2.5.1 Overview

The What-If Simulation lets farmers test how hypothetical future activities would affect their crop's health — without modifying any real data. You can add planned irrigations, fertilizer applications, or other activities and see the projected stress score, risk index, and growth stage.

### 2.5.2 Simulation Flow






**Endpoint:** `POST /api/v1/crops/{crop_id}/simulate`

**Request Body:**

```json
{
  "hypothetical_actions": [
    {
      "action_type": "irrigation",
      "effective_date": "2026-04-20",
      "category": "Operational"
    },
    {
      "action_type": "fertilizer",
      "effective_date": "2026-04-25",
      "category": "Operational"
    }
  ]
}
```

**Steps:**

1. Go to `/crops/[id]/simulate`
2. Click **Add Hypothetical Action** and add one or more future activities
3. Click **Run Simulation**
4. View the projected stress score, risk index, and growth stage
5. No actual crop data is modified

**Constraints:**

- Only the crop owner can run simulations
- Simulations cannot be run on Closed or Archived crops (returns `409 Conflict`)

---

## 2.6 Yield Submission

### 2.6.1 Overview

After marking a crop as harvested, farmers can submit their actual yield. The system compares the reported yield against a biologically estimated maximum and computes a verification score using ML-based heuristics.

verification score using ML-based heuristics.

**Endpoint:** `POST /api/v1/crops/{crop_id}/yield`

**Request Body:**

```json
{
  "reported_yield": 45.5,
  "yield_unit": "kg/acre",
  "harvest_date": "2026-04-10"
}
```

**Response Fields:**

| Field | Description |
|---|---|
| `reported_yield` | The yield you submitted |
| `ml_yield_value` | System-estimated yield for comparison |
| `biological_cap` | Maximum biologically plausible yield |
| `bio_cap_applied` | Whether the system applied the cap |
| `yield_verification_score` | Confidence score from 0.0 to 1.0 |

**Steps:**

1. Go to `/crops/[id]/yield`
2. Enter your reported yield (e.g., `45.5 kg/acre`)
3. Optionally set the harvest date
4. Click **Submit Yield**
5. View the verification score and comparison with the system estimate

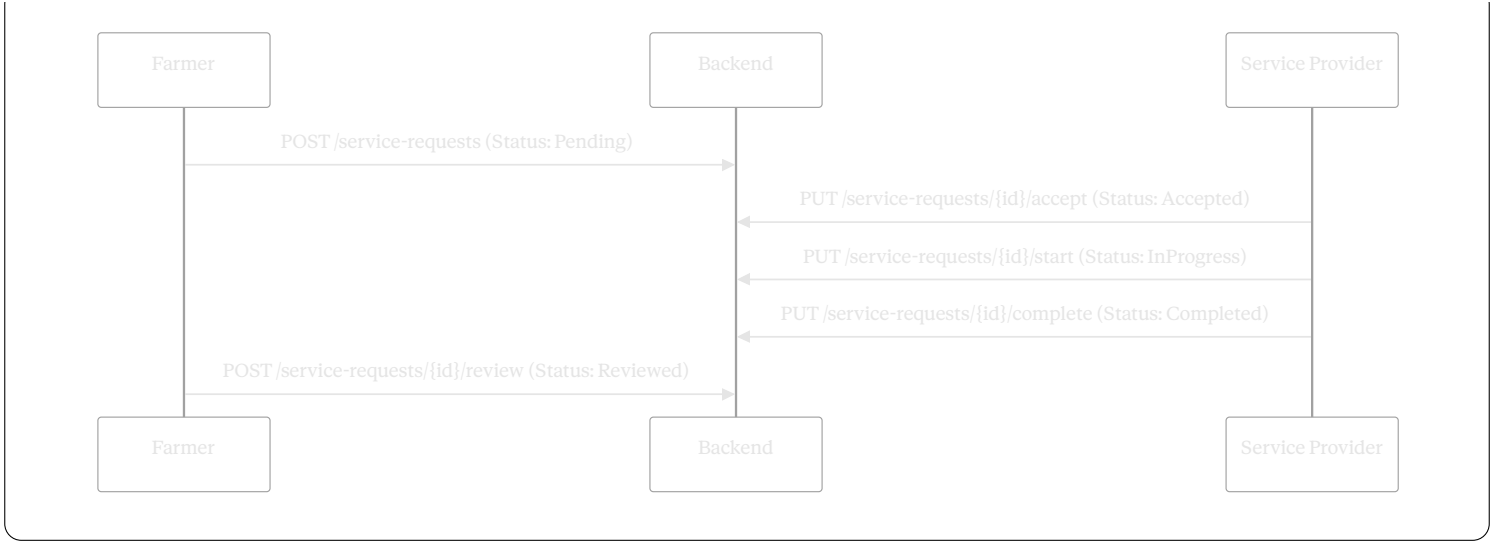
---

## 2.7 Service Marketplace (SOE)

### 2.7.1 Overview

The Service Orchestration Engine is a marketplace where farmers can book equipment rental, labor hire, and other agricultural services from registered providers. Each request moves through a clear lifecycle from booking to completion.

### 2.7.2 Full Service Request Lifecycle




### 2.7.3 Booking a Service (Farmer)

**Endpoint:** `POST /api/v1/service-requests/`

**Steps:**

1. Log in as Farmer
2. Go to `/services` → **Service Marketplace**
3. Browse available providers (`GET /api/v1/providers`)
4. Click a provider to view their profile, equipment, and ratings
5. Click **Request Service** (opens `/services/request`)
6. Fill in: service type, preferred date, description
7. Submit — request status becomes `Pending`
8. Track your requests at `/services/my-requests`

### 2.7.4 Responding to Requests (Provider)

- **Accept:** `PUT /api/v1/service-requests/{id}/accept` → status becomes `Accepted`
- **Decline:** `PUT /api/v1/service-requests/{id}/decline` → status becomes `Declined`

**Steps:**

1. Log in as Provider (`+919876543211`)
2. Go to `/provider` → **Provider Dashboard**
3. View incoming requests
4. Click **Accept** or **Decline**

### 2.7.5 Starting Work (Provider)

**Endpoint:** `PUT /api/v1/service-requests/{id}/start`

Moves the request from `Accepted` to `InProgress`.

### 2.7.6 Completing Work (Provider)

**Endpoint**: `PUT /api/v1/service-requests/{id}/complete`

Moves from `InProgress` to `Completed`.

### 2.7.7 Marking a Request as Failed (Provider)

**Endpoint**: `PUT /api/v1/service-requests/{id}/fail`

Used when work could not be completed (e.g., equipment breakdown, weather). Moves request to `Failed`.

### 2.7.8 Cancelling a Request

**Endpoint**: `PUT /api/v1/service-requests/{id}/cancel`

Both farmers and providers can cancel a request in `Pending` or `Accepted` status. Admins can cancel any request.

### 2.7.9 Reviewing a Completed Service (Farmer)

**Endpoint**: `POST /api/v1/service-requests/{id}/review`

**Body**: `{ "rating": 4, "comment": "Good service, arrived on time" }`

**Steps**:

1. Log in as Farmer
2. Go to `/services/my-requests`
3. Find a completed request
4. Click **Write Review**
5. Give a rating (1–5 stars) and optional comment
6. Submit

---

## 2.8 Equipment & Labor Management (Provider)

### 2.8.1 Managing Equipment

**Endpoints** (Provider only):

- `POST /api/v1/providers/{provider_id}/equipment/` — Add equipment
- `GET /api/v1/providers/{provider_id}/equipment/` — List equipment
- `PATCH /api/v1/providers/{provider_id}/equipment/{id}` — Update details
- `PATCH /api/v1/providers/{provider_id}/equipment/{id}/availability` — Toggle availability

- `DELETE /api/v1/providers/{provider_id}/equipment/{id}` — Remove

**Steps:**

1. Log in as Provider
2. Go to `/provider/equipment`
3. Click **Add Equipment**
4. Fill in: name, type, daily rate, description
5. Use the availability toggle to mark equipment as available or unavailable

### 2.8.2 Managing Labor

**Endpoints:**

- `POST /api/v1/labor/` — Add labor listing
- `GET /api/v1/labor/` — List listings
- `PUT /api/v1/labor/{id}` — Update
- `PATCH /api/v1/labor/{id}/availability` — Toggle availability
- `DELETE /api/v1/labor/{id}` — Remove

---

## 2.9 In-App Messaging

**Endpoints:**

- `GET /api/v1/messages/` — List conversations
- `POST /api/v1/messages/` — Send a message

**Steps:**

1. Go to `/messages`
2. Select an existing conversation or start a new one
3. Type your message and send

Messages support types: `text`, `system`, and `contact_share`. Read status (`is_read`, `read_at`) is tracked automatically.

---

## 2.10 Weather & Risk Assessment

**Endpoints:**

- `GET /api/v1/weather/?region=Punjab` — Weather risk for a region

- `GET /api/v1/weather/risk?crop_id={id}` — Crop-specific weather risk

**Steps:**

1. Go to `/weather`
2. View weather conditions and risk levels for your region
3. Weather risk values are automatically factored into crop stress calculations

---

## 2.11 Government Schemes

**Endpoints:**

- `GET /api/v1/schemes/` — List schemes
- `GET /api/v1/schemes/{id}` — Scheme details
- `POST /api/v1/schemes/{id}/redirect` — Track click-through to official portal

**Steps:**

1. Log in as Farmer
2. Go to `/schemes`
3. Browse available government schemes relevant to your crop and region
4. Click a scheme to view eligibility criteria and benefits
5. Click **Apply on Official Portal** to be redirected to the government site (click-through is tracked)

---

## 2.12 Alerts & Notifications

**Endpoints:**

- `GET /api/v1/alerts/` — List your alerts
- `PUT /api/v1/alerts/{id}/acknowledge` — Mark as acknowledged
- `POST /api/v1/alerts/acknowledge-bulk` — Bulk acknowledge

**Alert Types:** `WeatherWarning`, `StressAlert`, `RiskThresholdBreached`, `GovernanceAction`, `SystemNotification`

**Severity Levels:** `info`, `warning`, `critical`

**Steps:**

1. Go to `/alerts`

2. View all alerts sorted by urgency
3. Click **Acknowledge** to mark individual alerts as read
4. Use **Acknowledge All** for bulk operation

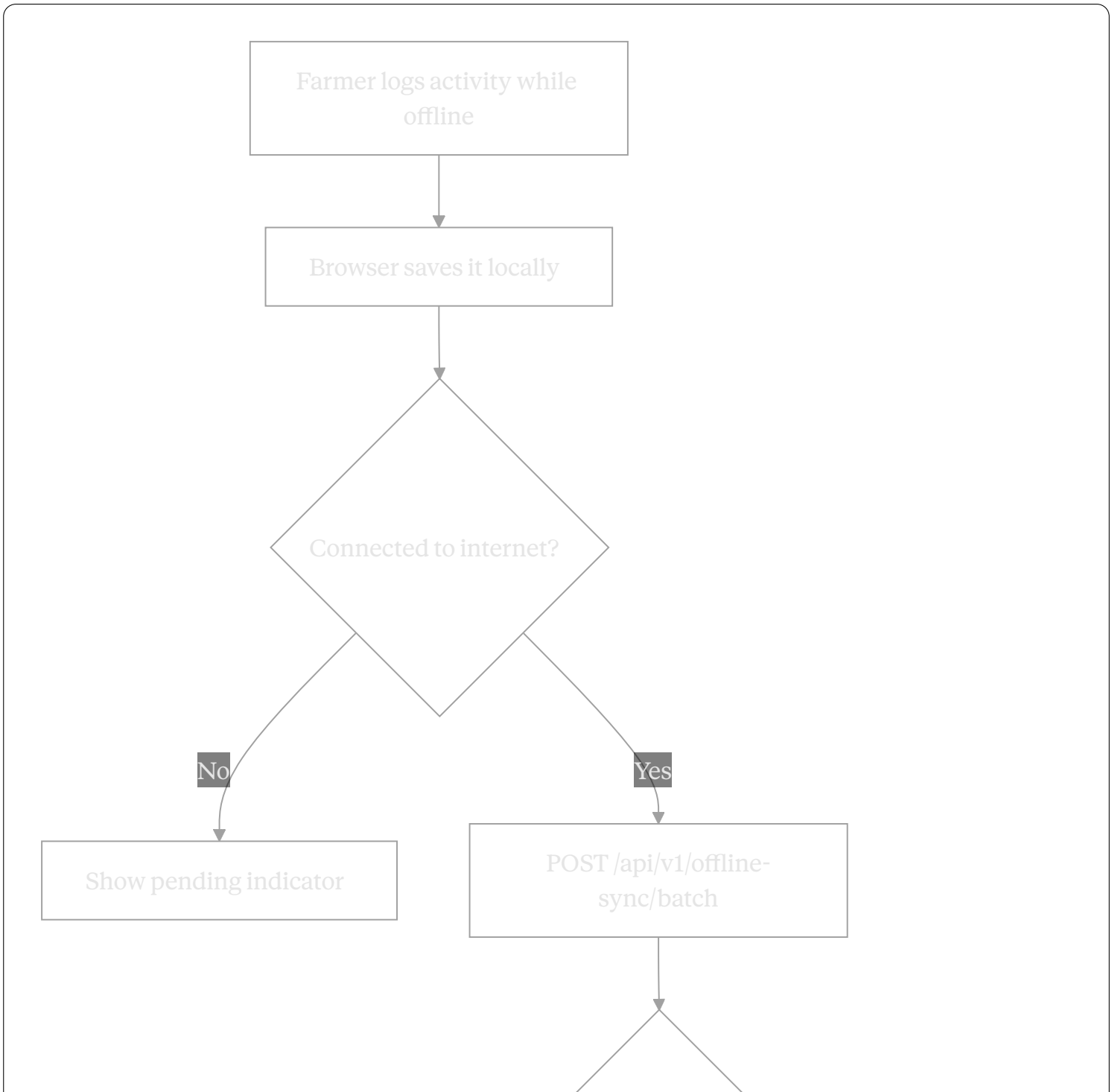
---

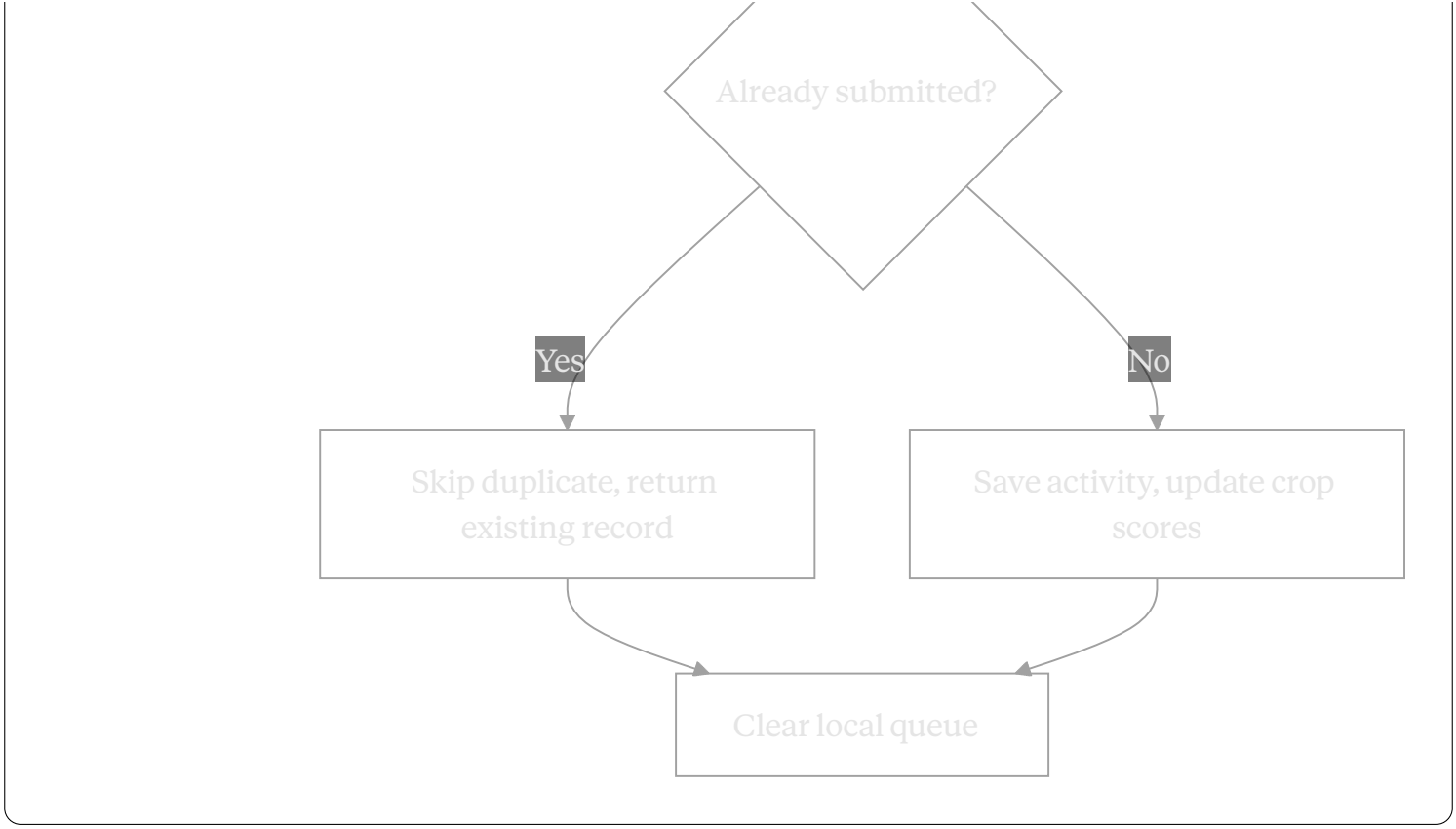
## 2.13 Offline Support (PWA)

### 2.13.1 Overview

CultivaX works fully offline. When you lose internet connectivity, activities you log are saved locally in the browser and automatically synced to the server once your connection is restored. Duplicate submissions are handled safely.

### 2.13.2 Offline Sync Flow






**Sync Endpoint:** `POST /api/v1/offline-sync/batch`

**Testing Steps:**

1. On a crop's activity page, open DevTools → Network → toggle **Offline**
2. Log an activity — the UI accepts it with a pending indicator
3. Toggle **Offline** off
4. The activity syncs automatically
5. Verify by checking `GET /api/v1/crops/{id}/actions/`

---

## 2.14 Admin — User Management

### 2.14.1 User Governance

**Endpoints** (Admin only):

- `GET /api/v1/admin/users` — List all users
- `PUT /api/v1/admin/providers/{id}/suspend` — Suspend a provider (requires a reason)
- `PUT /api/v1/admin/providers/{id}/unsuspend` — Unsuspend a provider

**Steps:**

1. Log in as Admin (`+919876543212`)
2. Go to `/admin/users` to view all registered users with roles, regions, and status

3. To suspend a provider: go to `/admin/providers`, select the provider, click **Suspend**, and enter a justification
4. The system logs the action in the audit trail and sends an alert to the provider

### 2.14.2 Provider Suspension Rules

| Action | Reason Required | Audit Logged | Alert Sent to Provider |
|---|---|---|---|
| Suspend | Yes (mandatory) | Yes | Critical alert |
| Unsuspend | Yes (mandatory) | Yes | Info alert |

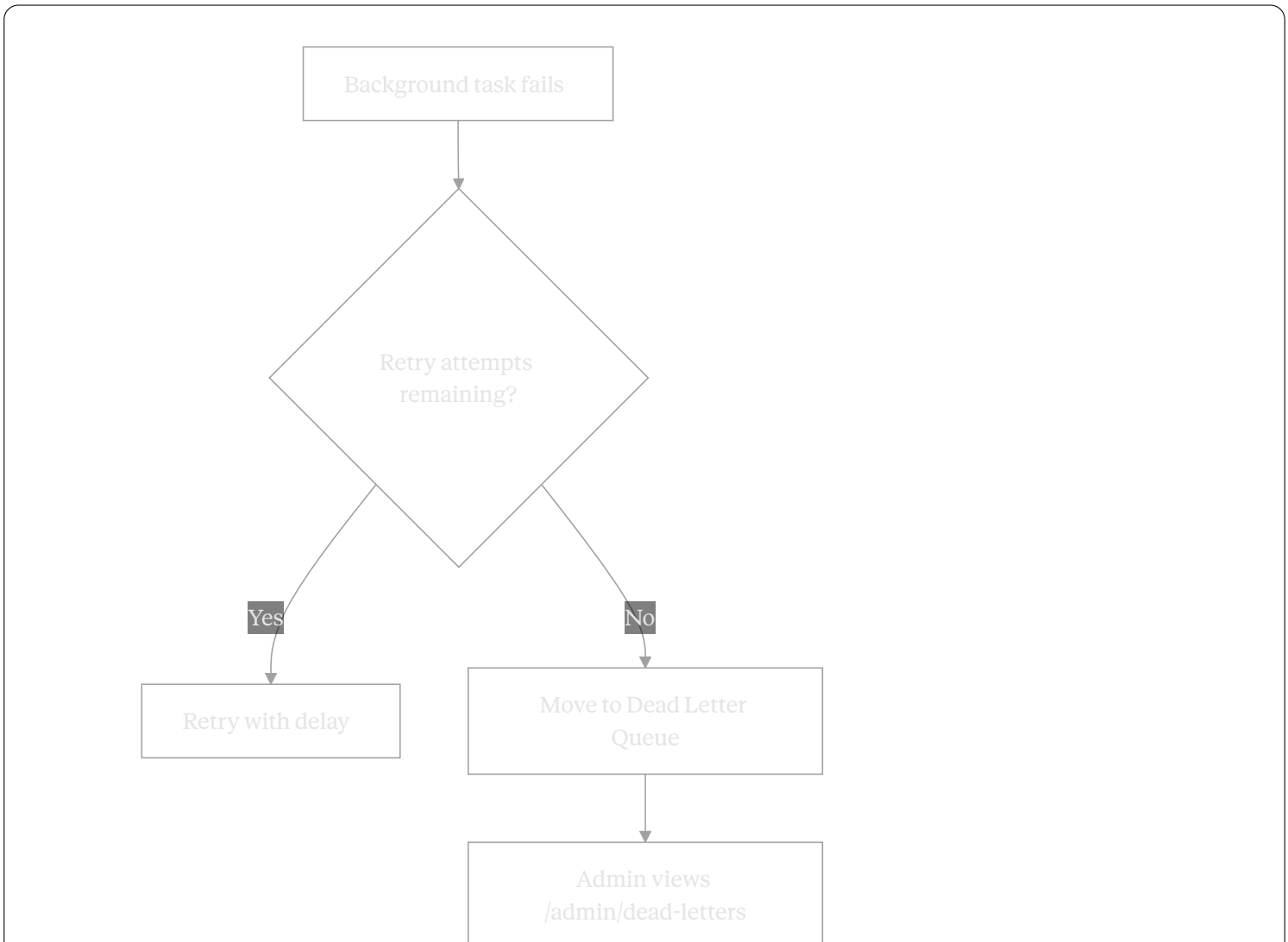
---

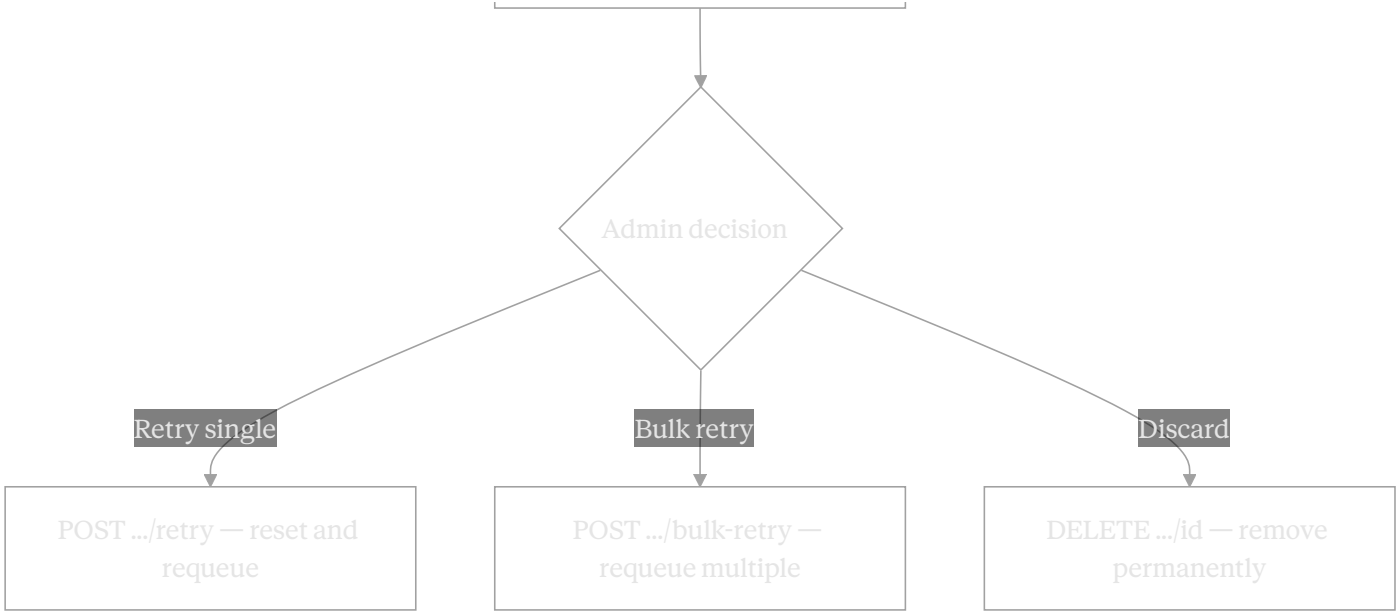
## 2.15 Admin — Dead Letter Queue (DLQ)

### 2.15.1 Overview

When a background processing task (such as a crop score recalculation) fails repeatedly, it is moved to the Dead Letter Queue. Admins can inspect these failures, retry them, or discard them.

### 2.15.2 DLQ Management Flow






**Endpoints:**

- `GET /api/v1/admin/dead-letters?page=1&per_page=50&event_type=ActionLogged` — List with filters
- `POST /api/v1/admin/dead-letters/{event_id}/retry` — Retry a single item
- `POST /api/v1/admin/dead-letters/bulk-retry` — Bulk retry with filters
- `DELETE /api/v1/admin/dead-letters/{event_id}` — Permanently discard

**Bulk Retry Request Body:**

```json
{
  "event_type": "ActionLogged",
  "older_than_minutes": 60,
  "limit": 100,
  "reason": "Transient database error resolved"
}
```

**Steps:**

1. Go to `/admin/dead-letters`
2. View failed tasks with their error messages, retry counts, and timestamps
3. Click **Retry** on individual items or use **Bulk Retry** with filters
4. All actions are recorded in the audit trail

---

## 2.16 Admin — Audit Trail

**Endpoint:** `GET /api/v1/admin/audit?page=1&per_page=20&action=provider_suspended`

**Available Filters:** `action`, `admin_id`, `entity_type`, `date_from`, `date_to`

Every administrative action — suspensions, retries, discards, flag reviews, rule changes — is permanently recorded with before/after values and justification notes.

**Steps:**

1. Go to `/admin/audit`
2. Browse all admin actions with timestamps and reasons
3. Filter by action type, admin, or date range

---

## 2.17 Admin — ML Model Management

**Endpoints:**

- `GET /api/v1/ml/models` — List registered models
- `GET /api/v1/ml/feedback` — View farmer feedback on predictions
- `GET /api/v1/ml/inference-audits` — Audit prediction decisions
- `GET /api/v1/ml/training-audits` — Audit training runs

**Steps:**

1. Go to `/admin/ml-models`
2. View all registered models with their versions and status
3. Check the feedback section to see how farmers rate ML predictions
4. Review the inference audit trail to see what predictions were made and why

---

## 2.18 Admin — Crop Rules Engine

**Endpoints:**

- `GET /api/v1/rules/` — List all crop rule templates
- `POST /api/v1/rules/` — Create a new rule (Admin only)

**Steps:**

1. Go to `/admin/rules`
2. View crop rule templates that define growth stages, recommended activities, and score penalties
3. Create or modify rules — all changes are version-tracked

---

## 2.19 Admin — Abuse Flag Pipeline

**Endpoints:**

- `GET /api/v1/admin/abuse-flags?status=open` — List flags
- `GET /api/v1/admin/abuse-flags/{id}` — View flag details
- `PATCH /api/v1/admin/abuse-flags/{id}/review?new_status=actioned` — Review a flag

**Steps:**

1. Go to the Admin Dashboard
2. View flagged accounts with severity scores and anomaly details
3. Review each flag and set its status to `reviewed`, `dismissed`, or `actioned`

---

## 2.20 Admin — System Health & Maintenance

**Endpoints:**

- `GET /api/v1/admin/health` — Full system health check with latency probes
- `GET /api/v1/admin/maintenance/status` — Scheduled task status and last run times
- `POST /api/v1/admin/maintenance/run?cadence=hourly&force=true` — Manually trigger scheduled tasks
- `POST /api/v1/admin/crops/{crop_id}/force-replay` — Queue an immediate score recalculation for a crop

**Steps:**

1. Go to `/admin/health` — view all subsystem statuses
2. Go to `/admin/maintenance` — view the scheduled task schedule
3. Click **Run Now** to manually trigger hourly, daily, or weekly maintenance during a demo

---

## 2.21 Analytics & Reporting

**Endpoints:**

- `GET /api/v1/analytics/overview` — Platform-level stats
- `GET /api/v1/analytics/crops/distribution` — Crop type distribution

- `GET /api/v1/analytics/regions/demand` — Regional demand analysis
- `GET /api/v1/analytics/admin-stats` — Admin-level statistics
- `GET /api/v1/dashboard/stats` — Farmer dashboard summary
- `GET /api/v1/dashboard/activity` — Recent activity feed

---

## 2.22 Search

**Endpoint**: `GET /api/v1/search/?q=wheat&page=1&per_page=20`

Full-text search across crops, providers, equipment, and services. Results are ranked by relevance score.

---

## 2.23 Account Management & Data Privacy

**Endpoints**:

- `GET /api/v1/account/me` — View your account details
- `GET /api/v1/account/me/export` — Export all your personal data (GDPR-style)
- `POST /api/v1/account/me/delete` — Request account deletion
- `GET /api/v1/consent/purposes` — View data consent purposes
- `POST /api/v1/consent/me/{purpose}` — Grant consent
- `DELETE /api/v1/consent/me/{purpose}` — Revoke consent

**Admin Data Export**: `GET /api/v1/admin/farmers/{farmer_id}/export` — Export a farmer's full data as JSON (crops, actions, media, alerts)

---

## 2.24 Language Settings (i18n)

**Endpoint**: `GET /api/v1/translations/{locale}` — Fetch UI strings for a locale

**Supported Languages**: `en` (English), `hi` (Hindi), `ta` (Tamil), `te` (Telugu), `mr` (Marathi)

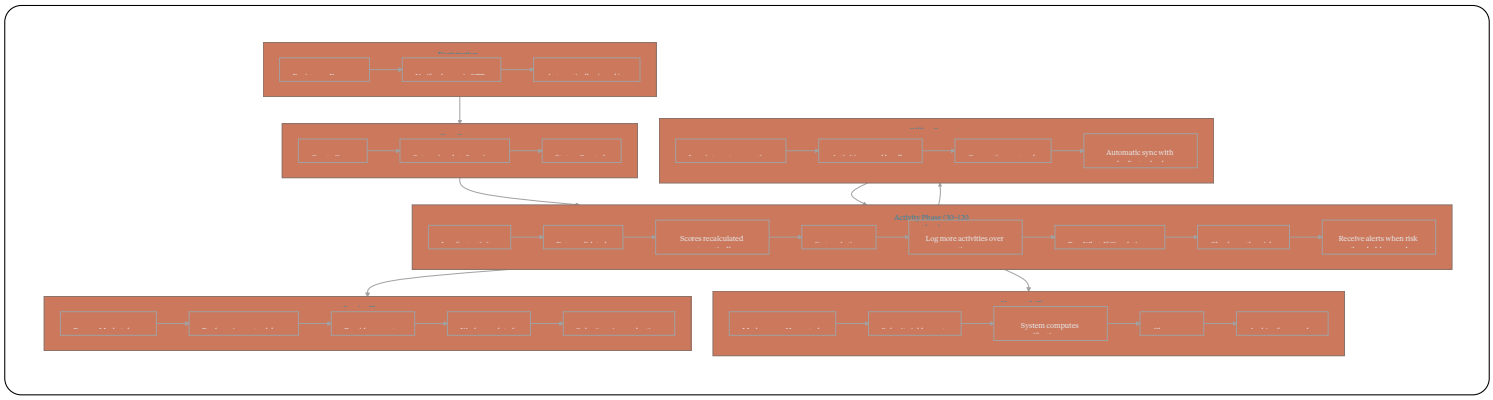
**Steps**:

1. Go to Settings
2. Change your preferred language
3. The entire interface switches to the selected language

---

## 2.25 Complete End-to-End User Journey

This diagram shows the full lifecycle of a farmer from registration through crop completion:




---

# APPENDIX A — Known Limitations & Demo Constraints

| # | Feature | Limitation | Workaround |
|---|---|---|---|
| 1 | **SMS / OTP Delivery** | SMS is fully mocked — OTPs are not actually sent. | The OTP is returned in the API response as `debug_otp` and also printed to the backend console: `docker compose logs backend` |
| 2 | **ML Inference** | No live TensorFlow/PyTorch models. Risk and stress scores use formula-based heuristics. | Scores are computed deterministically from the activity timeline using the built-in stress calculation engine |
| 3 | **Weather Data** | Weather values are rule-based, not from a live API (e.g., OpenWeather). | The `weather_risk` field in activity metadata allows manual injection of weather signals for testing |
| 4 | **Geospatial Mapping** | Land parcels use simple lat/lng coordinates with no polygon topology or satellite imagery. | Basic map views are rendered via Google Maps embeds |
| 5 | **Cron Jobs** | Background tasks (trust decay, stale data cleanup) run on 6–24 hour intervals. | Admin can manually trigger any maintenance cadence via `POST /api/v1/admin/maintenance/run?cadence=hourly&force=true` |
| 6 | **Media Analysis** | Image upload and analysis pipeline is stubbed — no computer vision processing. | Files upload and store metadata correctly, but `analysis_status` remains `pending` |
| 7 | **WhatsApp Integration** | WhatsApp webhook exists but is not connected to a live WhatsApp Business API | The endpoint validates webhook signatures correctly but operates in pass-through mode |

| | | | |
|---|---|---|---|
| | | WhatsApp Business API. | |
| 8 | **Payment Processing** | No payment gateway. Service requests do not involve real monetary transactions. | Pricing is tracked as metadata only |
| 9 | **Push Notifications** | Browser push notifications are not implemented. All notifications are in-app only. | Use the `/alerts` page to view all notifications |
| 10 | **Email Delivery** | No email provider configured. Password reset via email is not operational. | Use phone + OTP login instead |

# APPENDIX B — API Reference Summary

**Base URL:** `http://localhost:8000/api/v1`

**Interactive API Docs:** `http://localhost:8000/docs` (Swagger UI) · `http://localhost:8000/redoc` (ReDoc)

| Method | Endpoint | Auth Required | Role | Description |
|---|---|---|---|---|
| POST | `/auth/register` | No | — | Create account |
| POST | `/auth/login` | No | — | Password login |
| POST | `/auth/send-otp` | No | — | Request OTP |
| POST | `/auth/verify-otp` | No | — | Verify OTP and login |
| POST | `/auth/refresh` | Cookie | — | Rotate tokens |
| POST | `/auth/logout` | Cookie | — | End session |
| GET | `/auth/me` | Yes | — | Current user info |
| GET | `/auth/sessions` | Yes | — | List active sessions |
| POST | `/auth/sessions/revoke-all` | Yes | — | Revoke all other sessions |
| POST | `/crops/` | Yes | Farmer | Create crop |
| GET | `/crops/` | Yes | Farmer | List crops |
| GET | `/crops/{id}` | Yes | Farmer | Crop detail |

| | | | | |
|---|---|---|---|---|
| PUT | `/crops/{id}` | Yes | Farmer | Update crop |
| PUT | `/crops/{id}/sowing-date` | Yes | Farmer | Change sowing date (triggers full recalculation) |
| PUT | `/crops/{id}/activate` | Yes | Farmer | Move to Active |
| PUT | `/crops/{id}/harvest` | Yes | Farmer | Move to Harvested |
| PUT | `/crops/{id}/close` | Yes | Farmer | Move to Closed |
| PUT | `/crops/{id}/archive` | Yes | Farmer | Archive crop |
| PUT | `/crops/{id}/unarchive` | Yes | Farmer | Unarchive crop |
| POST | `/crops/{id}/actions/` | Yes | Farmer | Log activity |
| GET | `/crops/{id}/actions/` | Yes | Farmer | List activities |
| POST | `/crops/{id}/simulate` | Yes | Farmer | What-if simulation |
| POST | `/crops/{id}/yield` | Yes | Farmer | Submit yield |
| GET | `/crops/{id}/replay/status` | Yes | Farmer | Score recalculation status |
| GET | `/crops/{id}/replay/history` | Yes | Farmer | Recalculation event log |
| GET | `/crops/{id}/snapshots` | Yes | Farmer | List snapshots |
| POST | `/service-requests/` | Yes | Farmer | Create service request |
| GET | `/service-requests/` | Yes | All | List requests |
| PUT | `/service-requests/{id}/accept` | Yes | Provider | Accept request |
| PUT | `/service-requests/{id}/start` | Yes | Provider | Start work |
| PUT | `/service-requests/{id}/complete` | Yes | Provider | Mark completed |
| PUT | `/service-requests/{id}/decline` | Yes | Provider | Decline request |
| PUT | `/service-requests/{id}/cancel` | Yes | All | Cancel request |
| PUT | `/service-requests/{id}/fail` | Yes | Provider | Mark as failed |

| | | | | |
|---|---|---|---|---|
| POST | `/service-requests/{id}/review` | Yes | Farmer | Submit review |
| GET | `/providers/` | Yes | All | List providers |
| GET | `/providers/{id}` | Yes | All | Provider detail |
| GET | `/admin/dead-letters` | Yes | Admin | List failed tasks |
| POST | `/admin/dead-letters/{id}/retry` | Yes | Admin | Retry single task |
| POST | `/admin/dead-letters/bulk-retry` | Yes | Admin | Bulk retry |
| DELETE | `/admin/dead-letters/{id}` | Yes | Admin | Discard task |
| GET | `/admin/audit` | Yes | Admin | Audit trail |
| GET | `/admin/health` | Yes | Admin | System health |
| POST | `/admin/maintenance/run` | Yes | Admin | Trigger scheduled task |
| GET | `/admin/abuse-flags` | Yes | Admin | Abuse flags |
| GET | `/alerts/` | Yes | All | My alerts |
| PUT | `/alerts/{id}/acknowledge` | Yes | All | Acknowledge alert |
| GET | `/weather/` | Yes | Farmer | Weather data |
| GET | `/schemes/` | Yes | Farmer | Government schemes |
| GET | `/ml/models` | Yes | Admin | ML models |
| GET | `/search/?q=...` | Yes | All | Full-text search |
| GET | `/translations/{locale}` | No | — | Language strings |

## APPENDIX C — Running Automated Tests

To verify all automated tests pass:

```bash
cd backend
source venv/bin/activate
```

```
pytest -q --tb=short
```

**Expected Result:** `458 passed, 20 skipped, 0 failed`

The 20 skipped tests are asynchronous tests that require `pytest-asyncio` and are intentionally skipped in the standard synchronous test runner.